A3-PS036-TC095

PQC DNSSEC MTL Benchmark Using a Reference Implementation

Trushank
tv25csb0a37@student.nitw.ac.in

Nithik R.
nr25mae0009@student.nitw.ac.in

AIORI Hackathon 3 October 13, 2026

Problem Description

PQC signatures can enlarge DNSSEC responses and risk UDP fragmentation

Post-quantum cryptography (PQC) is intended to resist quantum-enabled cryptanalysis, but PQC public keys and signatures can be much larger than their classical counterparts. In DNSSEC, larger records can increase DNS response size, raising the risk of fragmentation, dropped responses, and slower resolution. PS-036 asks the team to benchmark a mentor-tested Merkle Tree Ladder (MTL) reference implementation for one supported PQC signature algorithm, using equivalent workloads with and without MTL where supported.

- **Response-size pressure:** PQC signatures and keys can push DNSSEC responses beyond practical UDP payload sizes.
- **Network impact:** oversized responses can fragment or be dropped, potentially increasing latency and lookup failures.
- **Performance trade-off:** MTL may reduce per-message transmitted signature overhead, but adds hashing, ladder maintenance, authentication-path processing, and cache dependencies.
- **Fair comparison:** results are meaningful only when workloads, algorithm, environment, and trial procedure are held consistent.
- **Scope boundary:** use one supported PQC algorithm and one mentor-tested MTL implementation; do not claim a new security proof or deploy on the public Internet.

Background

PQC signature sizes create DNS response pressure; MTL offers compact authentication paths.

DNSSEC authenticates DNS data using digital signatures. The PS-036 brief highlights a practical concern for post-quantum migration: larger PQC keys and signatures can increase DNS response sizes. PQC signatures as ranging from hundreds to thousands of bytes, with some schemes substantially larger; the exact size depends on the chosen algorithm and parameters.

MTL concept

Merkle Tree Ladder mode authenticates an evolving series of messages with a set of Merkle-tree roots (“rungs”) and compact authentication paths. The active tree structure is associated with the binary representation of the number of messages. A series identifier (SID) provides domain separation for node hashing.
At a high level, MTL’s workflow is:

- **Sign:** derive a message leaf, update the active node set, and sign the current ladder roots with the underlying signature scheme.
- **Condense:** transmit a compact authentication path and a handle that identifies a reference ladder value, rather than repeating all bulky signature material.
- **Reconstitute and verify:** the verifier uses the condensed signature and the referenced ladder; if the value is not cached, it may need to fetch it before normal MTL verification.

Literature & RFC Review

Key source: draft-harvey-cfrg-mtl-mode and the PS-036 / IETF 126 context.

1. IETF draft: draft-harvey-cfrg-mtl-mode

The notes describe MTL as a mode for authenticating a series of messages using a Merkle node set, ladder roots, and authentication paths. Its condensation/reconstitution mechanism is directly relevant to the PS-036 goal of comparing transmitted bytes. The notes also highlight backward compatibility: a message's authentication path relative to a newer ladder can be checked using an older authenticated ladder, subject to the draft's stated conditions.

2. DNSSEC and PQC context in the PS-036 brief

The problem statement frames the issue as the interaction between larger PQC signatures and DNS UDP response sizes. It requires one supported PQC signature algorithm, three dataset sizes, repeated trials, a pinned software/hardware environment, and comparisons with and without MTL where supported.

3. IETF 126 Hackathon

The PS-036 reference list cites the IETF 126 Hackathon. In this project, that reference is contextual: implementation details, fixtures, and the mentor-tested reference package supplied for the challenge take precedence over an independently re-created implementation.

Proposed Solution

Wrapping the supplied reference API with a repeatable C/C++ benchmark harness.

We will build a small C/C++ benchmark harness around the mentor-tested MTL reference implementation once it is. Keeping cryptographic operations inside the supplied implementation we will use the harness to prepare equivalent workloads, run trials, capture measurements, and produce reproducible output.

Workflow

1. **Pin the environment:** recording OS/build configuration, CPU, compiler, dependency versions, reference implementation revision, PQC algorithm/parameters, and relevant runtime settings.
2. **Validate fixtures:** compiling and running the mentor's smoke tests or examples unchanged before adding benchmark logic.
3. **Prepare datasets:** defining small, medium, and large message sets with documented sizes and deterministic generation rules.
4. **Run paired workloads:** benchmarking MTL and the equivalent non-MTL path where the reference implementation supports both.
5. **Repeat Trials:** separating warm-up from recorded runs; retaining raw observations rather than only averages.
6. **Collect metrics:** recording signing latency, verification time, transmitted bytes, and cache observations or clearly labelled estimates.
7. **Report:** exporting raw CSV/JSON, summarizing median and spread, document failures and deviations, and generating comparison charts

Optimization Proposed

Measuring the trade-off between compact transmission and added MTL processing.

The benchmark is designed to reveal trade-offs. Our optimization target is to reduce repeated transmission of bulky signature material where MTL supports it , While quantifying the added signer/verifier work on dependence on cached or separately distributed reference values. The table below states the expected comparison dimensions, not pre-measured results.

Dimension	Baseline (without MTL)	MTL path	How to compare
Signature material	Full signature/workload as supported by the reference	Condensed signature + handle/path and reference ladder	Count the same defined wire fields for both paths
Signing latency	Underlying PQC signing for each workload item	Hashing, node-set update, ladder signing and condensation as applicable	Same timer boundaries; repeated trials
Verification time	Baseline verification operation	Reconstitution, path checks and underlying verification as applicable	Report total and, if available, component timings
Transmitted bytes	Bytes for baseline signature-related response data	Bytes for condensed data plus handle; account for ladder distribution separately	State exactly what is included in byte counts
Cache usage	Baseline cache assumptions	Reference ladder cache footprint and hit/miss behavior	Measured values separate from estimates
Reproducibility	Pinned config and fixed workload	Same config and equivalent workload	Publish raw data, version details and run commands

Solution Architecture & Design

Workloads feed the harness, which runs baseline and MTL paths and collects metrics.

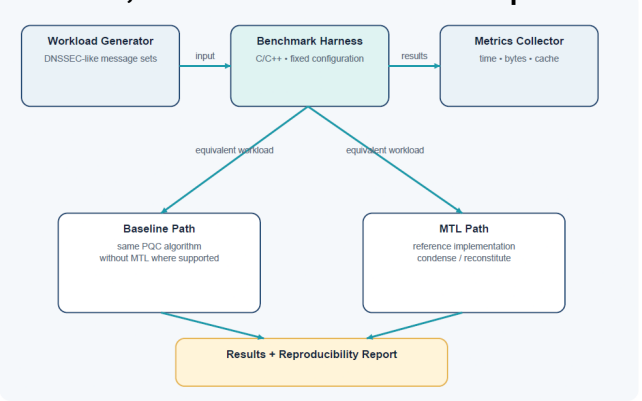


Figure: System Architecture UML Diagram

Data Flow Diagram

Export raw results, configuration, and cache labels for reproducibility.

This data-flow view shows how test inputs become benchmark runs and then auditable outputs.

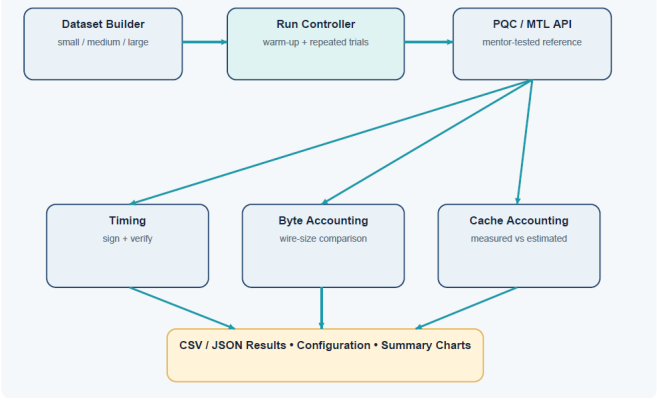


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Deployment Timeline and Methodology are as follows:

1. **Environment:** Creating repository, recording hardware/OS, choosing compiler/build system, adding reproducible scripts and configuring template. End goal is to build clean environment

2. **Reference integration:** Setting up the reference implementation in the environment and run the basics test to confirm it's working. End goal is to make the implementation running and document the versions and implementation.

3. **Harness adapter:** Implementing thin C/C++ adapter and validating one known example per supported path. End goal is to pass all the tests and record their outputs.

4. **Workloads:** Creating three dataset sizes and matched MTL/non-MTL cases where supported; defining warm-up and repetitions. End goal is to design workload supporting both MTL/non-MTL cases.

5. **Benchmark runs:** Running repeated trials; collect timing, byte counts, cache metrics/estimates; retaining raw logs. End goal is to aggregate the results for MTL/non-MTL cases against the workloads.

6. **Analysis & demo:** Generating summaries/plots, document limitations, creating pseudocode and mentor demonstration. End is to create the complete record and documentation benchmarking the implementation.

Bibliography I

[1] IETF, draft-harvey-cfrg-mtl-mode, Merkle Tree Ladder (MTL) mode. Referenced by the supplied PS-036 problem statement .https://datatracker.ietf.org/doc/draft-harvey-cfrg-mtl-mode/

[2] AIORI Hackathon 3, PS-036: PQC DNSSEC MTL Benchmark Using a Reference Implementation, supplied problem statement, October 2026.

[3] IETF 126 Hackathon. Listed as a reference/context item in the supplied PS-036 problem statement. https://wiki.ietf.org/en/meeting/126/hackathon

Thank You for Your Attention!